

Curso de Especialización Ciberseguridad en las TIC

Proyecto Hardening Ubuntu Server. Hardening de procesos

Objetivos

- Comprender cómo funciona Curl y AppArmor.
- Saber crear perfiles en AppArmor correctamente.
- Realizar la fortificación del servicio Apache2 mediante AppArmor.

Resumen

Autora: Marina del Pilar López Oliva

Enero-Febrero 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	30/01/2026	Documentación del apartado A y la introducción de la práctica.
1.1	01/02/2026	Documentación del apartado B y la conclusión de la práctica.

Índice de Contenidos

1. Introducción.....	4
2. Apartado A.....	4
3. Apartado B.....	6
4. Conclusiones.....	8
5. Webgrafía.....	9

1. Introducción

La protección de servicios es una prioridad crítica. En esta práctica nos centramos en la fortificación del servidor Ubuntu Server que contiene la web de nuestro subsistema. Utilizamos AppArmor, un módulo de seguridad del kernel de Linux que permite el control de acceso obligatorio. En el primer apartado, abordaremos primero la comprensión teórica del funcionamiento de los perfiles y su interacción con los binarios del sistema. En el siguiente apartado, ejecutaremos un enjaulamiento práctico del servicio Apache2. El objetivo final de este documento es demostrar cómo limitar el radio de acción de un proceso puede prevenir la explotación de vulnerabilidades y accesos no autorizados a información sensible.

2. Apartado A.

Basándonos en el ejemplo práctico del capítulo 1.7.2.1 de los contenidos de la Ut4, tenemos que contestar las siguientes preguntas.

A. ¿Cuántos ficheros tiene que leer curl para poder ejecutarse?.

Curl necesita leer varios ficheros para poder funcionar correctamente. Ya que necesita acceso a archivos de configuración y a algunos recursos del sistema.

En el caso del ejemplo práctico, se mencionan tres archivos: `/etc/ssl/openssl.cnf`, `/etc/nsswitch.conf` y también `/etc/passwd`, aunque este último no lo necesita para funcionar.

Cuando se realiza el proceso de “aa-logprof” podemos observar que pide acceso a `/usr/bin/touch` que se le deniega, por lo que no lo necesita para funcional. Además, pide acceso a su propio directorio `/usr/bin/curl`.

Para un funcionamiento correcto y completo, también necesitaría acceso a otros archivos como a las librerías estándares del sistema para SSL y posibles archivos de configuración adicionales.

B. ¿Sabes para qué sirve cada uno de ellos?.

Curl necesita leer el archivo `/etc/openssl.cnf` para poder resolver la IP de la dirección `www.google.es`. Permite establecer conexiones HTTPS seguras con servidores web.

El `nsswitch.conf`, este archivo es utilizado para determinar las fuentes de las que debe obtener la información y el nombre de servicio en una variedad de categorías y en qué orden.

El archivo `/etc/passwd` es donde se encuentran todos los usuarios del sistema. A través del cual se puede conocer todos los usuarios del sistema y los servicios que se están ejecutando en el servidor. Aunque en principio no es algo que necesite para funcionar correctamente.

Adicionalmente, se utilizarían archivos como `/etc/resolv.conf` para la resolución de IPs, `/etc/hosts` para la resolución local, y `/etc/ssl/certs` para certificados CA. También tendría que tener acceso a librerías por dependencias del sistema.

Además, en el mismo capítulo de los contenidos, hay un apartado de creación de un perfil para fortificar el comando `mycopy` donde permite la lectura de ficheros que hay en el directorio `ciber1` y la escritura de estos en el directorio `ciber2`. Se nos pregunta:

- A. ¿Qué debes modificar en el perfil para que permita la lectura completa del directorio `ciber1` (cualquier fichero) y la escritura en `ciber2` de cualquier fichero?.

Para que permita la lectura completa del directorio `ciber1` y la escritura en `ciber2` hay que tener en cuenta que se permita la ejecución de `mycopy` (`mr`), que se permita la lectura en los ficheros de `ciber1` (`r`), y la escritura en los del directorio `ciber2` (`w`).

Por lo que, para permitir la lectura completa del directorio `ciber1` y escritura en `ciber2` de cualquier fichero, debemos modificar el perfil reemplazando las reglas específicas por:

```
owner /home/*/ciber1/** r,
```

```
owner /home/*/ciber2/** w,
```

El `*` es un operador en AppArmor que coincide a un solo nivel de directorio. El `**` es otro operador de AppArmor que coincide a múltiples niveles de directorio.

3. Apartado B.

Una vez hemos comprendido cómo funciona AppArmor, vamos a realizar un enjaulamiento. En este apartado hemos configurado el sistema de control de acceso AppArmor con el objetivo de reforzar la seguridad del servidor web mediante el enjaulamiento de procesos.

Primero hemos procedido a actualizar los repositorios de ubuntu server y hemos instalado las herramientas necesarias para la gestión de perfiles de AppArmor.

```
admin@iescamp-webserver:/$ sudo apt install apparmor apparmor-profiles apparmor-utils
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
apparmor ya está en su versión más reciente (4.0.1really4.0.1-0ubuntu0.24.04.5).
apparmor-profiles ya está en su versión más reciente (4.0.1really4.0.1-0ubuntu0.24.04.5).
apparmor-utils ya está en su versión más reciente (4.0.1really4.0.1-0ubuntu0.24.04.5).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 13 no actualizados.
```

Figura 1. Instalación de paquetes de AppArmor y sus utilidades

Después hemos creado una estructura diferenciada dentro de la ruta raíz del servidor web. Definimos los directorios “Publico” y “Privado”, y les hemos asignado unos contenidos que muestren que es una página pública y que es una página privada. Para validar posteriormente la efectividad de las reglas de confinamiento.

```
admin@iescamp-webserver:/$ sudo mkdir /var/www/html/Publico
admin@iescamp-webserver:/$ sudo mkdir /var/www/html/Privado
admin@iescamp-webserver:/$ sudo cp /var/www/html/iescamp_mylocal/public_html/index.html /var/www/html/Publico/
admin@iescamp-webserver:/$ sudo cp /var/www/html/iescamp_mylocal/public_html/index.html /var/www/html/Privado/
```

Figura 2. Creación de directorios y creación del index.html

A continuación, hemos generado un perfil de seguridad para /usr/sbin/apache2. Para esto hemos seguido las instrucciones de los contenidos 1.7.2.1. Dentro del archivo, hemos aplicado restricciones de lectura específicas, destacando el uso de la directiva “deny” para bloquear cualquier intento de acceso al directorio “Privado”. En la siguiente figura se ve la modificación de “/etc/apparmor.d/usr.sbin/apache2”.

```

/var/www/html/ r,
/var/www/html/Publico/ r,
/var/www/html/Publico/** r,

# Enjaulamiento: Bloqueamos el directorio Privado
deny /var/www/html/Privado/ r,
deny /var/www/html/Privado/** r,
}

```

Figura 3. Configuración del perfil con restricciones de enjaulamiento

Una vez definido el perfil, hemos procedido a cargarlo en el kernel y establecer el modo “Enforce”. Con esto garantizamos que cualquier desviación de la política de seguridad sea bloqueada inmediatamente por el sistema, en lugar de ser solo notificada.

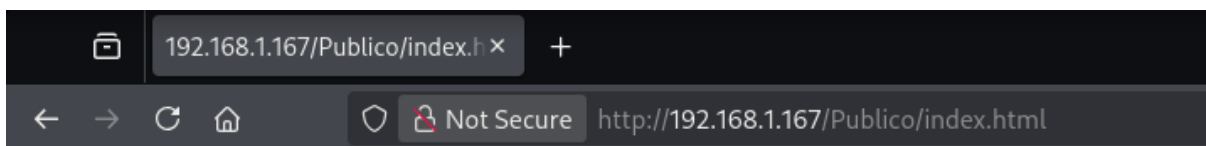
```

admin@iescamp-webserver:~$ sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.apache2
admin@iescamp-webserver:~$ sudo aa-enforce apache2
Setting /usr/sbin/apache2 to enforce mode.
Warning: profile apache2 represents multiple programs

```

Figura 4. Carga del perfil y establecimiento del modo estricto de ejecución

Por último, realizamos las pruebas de acceso. Primero vamos a probar que se puede acceder al recurso permitido. Como podemos observar, el servidor responde correctamente permitiendo la visualización del contenido.



Directorio PÚBLICO - Acceso Permitido

Figura 5. Acceso exitoso al directorio Público

Luego hemos verificado que al intentar acceder al recurso restringido, AppArmor intercepta la petición. El servidor devuelve un error de acceso denegado. Con esto demostramos que el proceso Apache2 está confinado y no puede “salir” de su jaula para leer archivos no autorizados.

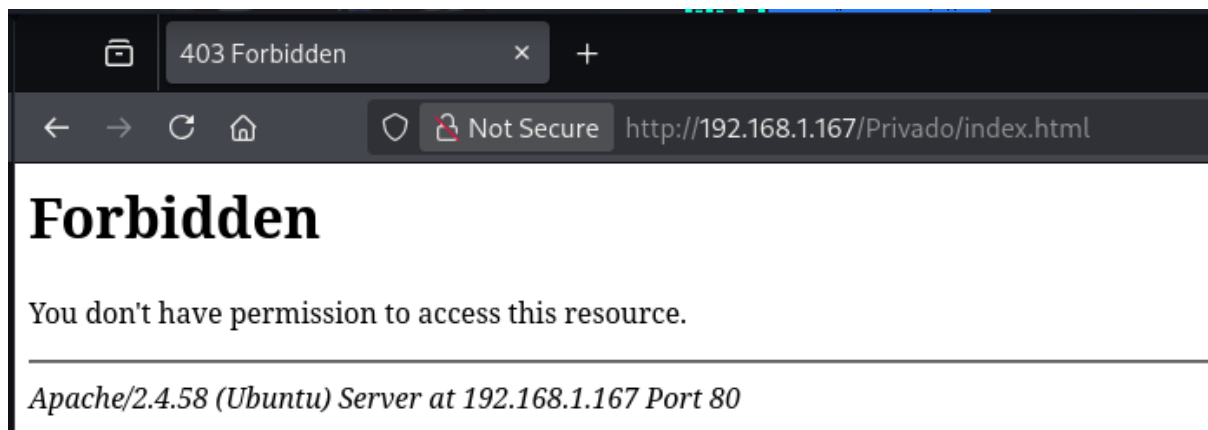


Figura 6. Bloqueo de seguridad por AppArmor al intentar acceder al Privado

Finalmente, consultamos los logs del kernel para verificar las trazas de seguridad. En los logs podemos ver las líneas ALLOWED para el tráfico legítimo y las DENIED que confirman que AppArmor ha bloqueado los intentos de acceso al directorio Privado.

```
[36392.471413] audit: type=1400 audit(1769970465.192:1876): apparmor="DENIED" operation="open" class="file" profile="/usr/sbin/apache2" name="/usr/share/zoneinfo/Etc/UTC" pid=391085 comm="a
pache2" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36406.331017] audit: type=1400 audit(1769970479.060:1879): apparmor="DENIED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/ld.so.cache" pid=391105 comm="apache2" req
uested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36406.336567] audit: type=1400 audit(1769970479.060:1880): apparmor="DENIED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/mime.types" pid=391105 comm="apache2" requ
ested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36406.336582] audit: type=1400 audit(1769970479.060:1881): apparmor="DENIED" operation="open" class="file" profile="/usr/sbin/apache2" name="/usr/share/zoneinfo/Etc/UTC" pid=391105 comm="a
pache2" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36417.574683] audit: type=1400 audit(1769970490.303:1884): apparmor="DENIED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/ld.so.cache" pid=391124 comm="apache2" req
uested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36417.580093] audit: type=1400 audit(1769970490.303:1885): apparmor="DENIED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/mime.types" pid=391124 comm="apache2" requ
ested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36417.580108] audit: type=1400 audit(1769970490.303:1886): apparmor="DENIED" operation="open" class="file" profile="/usr/sbin/apache2" name="/usr/share/zoneinfo/Etc/UTC" pid=391124 comm="a
pache2" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36444.340419] audit: type=1400 audit(1769970525.177:1888): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/ld.so.cache" pid=391139 comm="apache2" re
quested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36444.341133] audit: type=1400 audit(1769970525.177:1889): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/ld.so.cache" pid=391139 comm="apache2" re
quested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36444.345925] audit: type=1400 audit(1769970525.177:1890): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/mime.types" pid=391139 comm="apache2" req
uested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36444.347495] audit: type=1400 audit(1769970525.177:1891): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/ld.so.cache" pid=391139 comm="apache2" re
quested_mask="r" denied_mask="r" fsuid=0 ouid=0
[36444.349098] audit: type=1400 audit(1769970525.177:1892): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/dev/null" pid=391140 comm="apache2" requested
_mask="r" denied_mask="r" fsuid=0 ouid=0
[36444.352664] audit: type=1400 audit(1769970525.190:1893): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/dev/null" pid=391140 comm="apache2" requested
_mask="wc" denied_mask="wc" fsuid=0 ouid=0
[36444.352931] audit: type=1400 audit(1769970525.192:1894): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/dev/null" pid=391140 comm="apache2" requested
_mask="wc" denied_mask="wc" fsuid=0 ouid=0
[36444.353491] audit: type=1400 audit(1769970525.192:1895): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/etc/mime.types" pid=391140 comm="apache2" req
uested_mask="r" denied_mask="r" fsuid=33 ouid=0
[36444.354511] audit: type=1400 audit(1769970525.192:1896): apparmor="ALLOWED" operation="mknod" class="file" profile="/usr/sbin/apache2" name="/run/apache2/apache2.pid.N03qP9" pid=391140 c
omm="apache2" requested_mask="c" denied_mask="c" fsuid=0 ouid=0
[36451.118123] audit: type=1400 audit(1769970531.956:1913): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/usr/share/zoneinfo/Etc/UTC" pid=391142 comm="
apache2" requested_mask="r" denied_mask="r" fsuid=33 ouid=0
[36458.186588] audit: type=1400 audit(1769970547.041:1914): apparmor="ALLOWED" operation="open" class="file" profile="/usr/sbin/apache2" name="/usr/share/zoneinfo/Etc/UTC" pid=391143 comm="
apache2" requested_mask="r" denied_mask="r" fsuid=33 ouid=0
```

Figura 7. Registro de logs con eventos de denegación y permisividad

4. Conclusiones.

La realización de esta práctica, nos ha permitido constatar la eficacia de AppArmor como capa de defensa profunda.

Un aspecto clave durante el desarrollo fue la resolución de conflictos entre Apache2 y las restricciones del perfil. Al principio, el servidor web nos presentó fallos en el arranque debido a la denegación de acceso a librerías críticas de mod_ssl y archivos de logs que no estaban contemplados en la política inicial. Al final, pudimos resolver este conflicto y completar el enjaulamiento con éxito.

Con este proyecto hemos confirmado que es posible aislar directorios públicos de privados a nivel de kernel, impidiendo que una potencial brecha en el servidor web comprometa la integridad de los datos corporativos.

5. Webgrafía.

Punto 1.7.2.1 de los contenidos UT4.

Manual del curl incluido en Linux.

[[Página oficial del curl](#)]

[[Manual de nsswitch.conf](#)]

[[Manual AppArmor](#)]